SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 1

Design and Develop Modal

COURSE NAME: Deep Learning **COURSE CODE: MLA0408**

COURSE OUTCOME COVERED

CO 1: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems.. (BL-6)

Assessment Tool Weightage: 30%

QUESTIONS: **Total Marks:25**

Q1. Transfer Learning

Design a transfer learning framework for a plant disease detection system using a pre-trained CNN model. Justify the choice of layers to freeze and fine-tune, and propose suitable data augmentation techniques to improve classification performance.

Q2. DenseNet and PixelNet

Develop a CNN-based architecture by integrating DenseNet and PixelNet concepts for semantic image segmentation in autonomous driving. Explain how dense connectivity and pixel-wise prediction improve the overall segmentation accuracy.

Q3. Bidirectional RNN and Encoder–Decoder

Create an encoder–decoder sequence-to-sequence model using Bidirectional RNNs for multilingual machine translation. Illustrate the architecture and explain how bidirectional context enhances translation quality.

Q4. BPTT and LSTM

Design an LSTM-based network to predict stock market trends from historical time-series data. Explain how Backpropagation Through Time (BPTT) is applied during training and justify why LSTM is preferred over a standard RNN.

Q5. Integrated Deep Learning Model

Develop a deep learning solution for automatic video caption generation by integrating transfer learning for feature extraction with an LSTM-based encoder–decoder architecture. Justify the role of each component and explain how the complete system generates meaningful captions.

Code of Conduct:

I Haresh Kumar N L (192425009) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Answers:

1) Transfer Learning – Plant Disease Detection

A transfer learning framework can be designed for **plant disease detection** using a pre-trained CNN such as **VGG16, ResNet50, or MobileNet**. The pre-trained model has already learned useful features such as edges, shapes, textures, and patterns from a large image dataset.

Proposed Architecture

Input Image → Preprocessing → Pre-trained CNN → Feature Extraction → Fully Connected Layer → Softmax → Disease Class

Process

1. Collect images of healthy and diseased plant leaves.
2. Resize all images to a fixed size such as **224 × 224 pixels**.
3. Apply data augmentation to increase the variety of training images.
4. Load a pre-trained CNN model such as **VGG16** without its original classification layer.
5. Freeze the initial convolutional layers because they learn general features such as edges and textures.
6. Fine-tune the later convolutional layers because they can learn plant-specific features.
7. Add a new classification layer according to the number of plant disease classes.
8. Train the model using the plant disease dataset.
9. Evaluate the model using accuracy, precision, recall, and F1-score.

Layers to Freeze and Fine-Tune

- **Freeze:** Initial convolutional layers because their features are general and reusable.
- **Fine-tune:** Later convolutional layers because they need to adapt to leaf patterns and disease symptoms.
- **Train:** Newly added fully connected/classification layers.

Data Augmentation

Suitable techniques include:

- Rotation
- Horizontal/vertical flipping
- Zooming
- Random cropping
- Brightness adjustment

- Width and height shifting

These techniques reduce overfitting and improve the model's ability to recognize diseases under different conditions.

2) DenseNet and PixelNet – Autonomous Driving

A CNN architecture can be developed by combining **DenseNet** for feature extraction with **PixelNet** concepts for pixel-wise semantic segmentation. The system can identify objects such as **roads, vehicles, pedestrians, buildings, and traffic signs** in an autonomous driving scene.

Proposed Architecture

Input Road Image → DenseNet Feature Extraction → Dense Feature Maps → Pixel-wise Prediction → Segmentation Map

DenseNet Component

DenseNet connects each layer to all subsequent layers within a dense block.

This provides:

- Better feature reuse
- Improved gradient flow
- Reduced redundant feature learning
- Stronger representation of image features

For example, early layers may detect edges while later layers use these features to identify objects and road structures.

PixelNet Component

PixelNet performs prediction at the **pixel level**. Each pixel is assigned a class label.

Pixel Region	Class
Road pixels	Road
Car pixels	Vehicle
Human pixels	Pedestrian
Building pixels	Building
Sky pixels	Sky

Working

1. Input image is given to the network.
2. DenseNet extracts hierarchical visual features.

3. Dense connectivity allows features from earlier layers to be reused.
4. Feature maps are passed to the pixel-wise prediction component.
5. Each pixel is classified into a semantic category.
6. The final output is a colored segmentation map.

3) Bidirectional RNN + Encoder–Decoder – Machine Translation

A multilingual machine translation system can be developed using an **Encoder–Decoder architecture with Bidirectional RNNs**. The encoder reads the input sentence, while the decoder generates the translated sentence.

Architecture

Input Sentence → Bidirectional RNN Encoder → Context Representation → RNN Decoder → Translated Sentence

Example:

English:
I am going to school

French:
Je vais à l'école

Encoder

The Bidirectional RNN processes the input sequence in **two directions**:

- Forward direction: first word → last word
- Backward direction: last word → first word

The hidden representation therefore contains information from both past and future words.

Decoder

The decoder receives the encoded representation and generates the translated sentence **one word at a time**.

For example:

Je → vais → à → l'école

Advantages of Bidirectional Context

A normal RNN mainly processes information sequentially. A Bidirectional RNN considers information from both directions.

This helps the model:

- Understand the complete context of a sentence.
- Handle ambiguous words better.
- Capture relationships between distant words.

- Improve translation quality.

Working

1. Tokenize the input sentence.
2. Convert words into embeddings.
3. Pass embeddings through the Bidirectional RNN encoder.
4. Generate a context representation.
5. Pass the context to the decoder.
6. Decoder predicts the next translated word.
7. Continue until the end-of-sentence token is generated.

4) BPTT and LSTM – Stock Market Trend Prediction

An **LSTM-based neural network** can be used to predict stock market trends from historical time-series data. The input can contain information such as **opening price, closing price, highest price, lowest price, and trading volume**.

Architecture

Historical Stock Data → Preprocessing → LSTM Layers → Dense Layer → Trend Prediction

Example

Input:

Previous 10 days of stock data

Output:

Next-day trend → Up / Down

Why LSTM is Preferred Over Standard RNN

Standard RNNs have difficulty remembering information over long sequences because of the **vanishing gradient problem**.

LSTM solves this using:

- Forget gate
- Input gate
- Output gate
- Cell state

These components allow LSTM to remember important information for longer periods.

BPTT – Backpropagation Through Time

BPTT is used to train the LSTM network.

The process is:

1. Historical stock data is provided to the LSTM.
2. The LSTM processes the sequence step by step.
3. A prediction is generated.
4. The prediction is compared with the actual value.
5. The error/loss is calculated.
6. The error is propagated backward through the time steps.
7. Network weights are updated using an optimization algorithm such as Adam.
8. The process is repeated for multiple epochs.

Advantages of LSTM

- Handles long-term dependencies.
- Reduces the vanishing-gradient problem.
- Suitable for sequential/time-series data.
- Can remember important historical patterns.

5) Integrated Deep Learning Model – Video Caption Generation

An automatic video caption generation system can be developed by combining **transfer learning for video feature extraction** with an **LSTM encoder–decoder** for generating captions.

Architecture

Input Video → Frame Extraction → Pre-trained CNN → Feature Extraction → LSTM Encoder → LSTM Decoder → Text Caption

1. Video Input

The system receives a video as input.

Example:

A person is playing football.

2. Frame Extraction

Important frames are extracted from the video at regular intervals.

3. Transfer Learning

A pre-trained CNN such as **VGG16 or ResNet50** extracts visual features from each frame.

The CNN can identify:

- Objects
- People

- Background
- Visual patterns

The original classification layer is removed because the CNN is being used as a **feature extractor**.

4. LSTM Encoder

The extracted frame features are given to an LSTM encoder.

The encoder learns the sequence of events occurring in the video.

5. LSTM Decoder

The decoder converts the encoded visual information into words.

For example:

<START> → A → man → is → playing → football → <END>

Role of Each Component

Component	Role
Video	Provides visual input
Frame Extraction	Converts video into individual frames
Pre-trained CNN	Extracts visual features
LSTM Encoder	Understands temporal sequence
LSTM Decoder	Generates words sequentially
Output	Meaningful video caption

Complete Working

1. Input video is provided.
2. Video is divided into frames.
3. CNN extracts features from each frame.
4. Features are arranged according to their temporal order.
5. LSTM encoder learns the sequence.
6. LSTM decoder generates the caption word by word.
7. The generated words are combined to form the final sentence.

Example:

Input: Video showing a boy riding a bicycle.

Generated Caption:
A boy is riding a bicycle.